

OBJECTTM
TECHNOLOGIES

Java IO

www.object.co.in

What will be covered

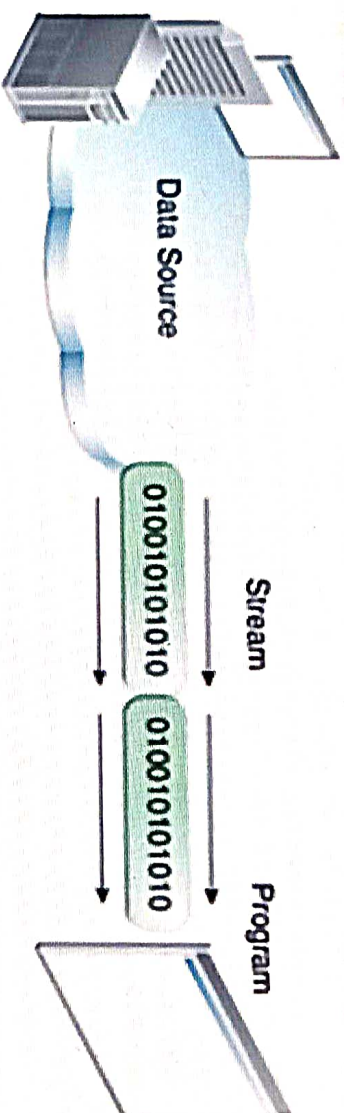
OBJECT

- What is stream
- IO Stream and its types
 - Byte Streams
 - Character Streams
 - Standard - built-in streams
 - Reading from console
 - File streams
 - File class
 - Reading data from file
 - Writing data to file

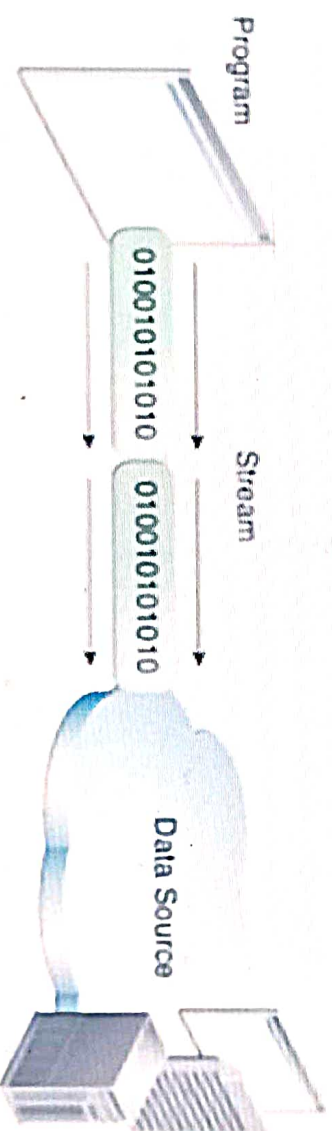


- An *I/O Stream* represents an input source or an output destination.
- A stream can represent many different kinds of sources and destinations, including disk files, devices, other programs, and memory arrays.
- Streams support many different kinds of data, including simple bytes, primitive data types, localized characters, and objects. Some streams simply pass on data; others manipulate and transform the data in useful ways.
- All streams present the same simple model to programs that use them: A stream is a sequence of data.
- A program uses an *input stream* to read data from a source, one item at a time. A program uses an *output stream* to write data to a destination, one item at time.

- Reading the data from the source

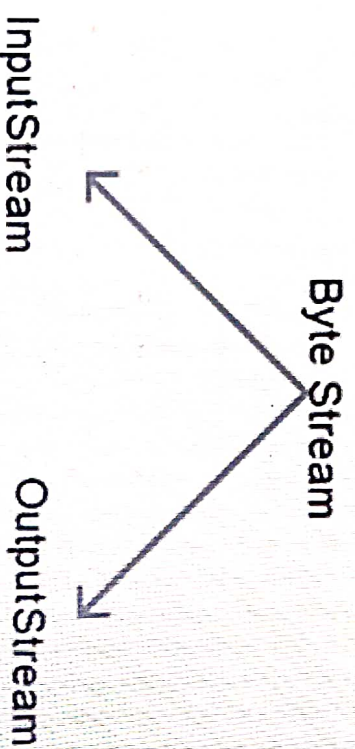


- Writing the data to the destination



- Java performs I/O through **Streams**. A Stream is linked to a physical layer by java I/O system to make input and output operation in java.
- Streams are clean way to deal with input/output without having every part of your code understand the physical.
- Java encapsulates Stream under **java.io** package. Java defines two types of streams. They are,
 - ◆ **Byte Stream** : It provides a convenient means for handling input and output of byte.
 - ◆ **Character Stream** : It provides a convenient means for handling input and output of characters. Character stream uses Unicode and therefore can be internationalize

- Byte stream is defined by using two abstract class at the top of hierarchy, they are `InputStream` and `OutputStream`
- These two abstract classes have several concrete classes that handle various devices such as disk files, network connection etc.



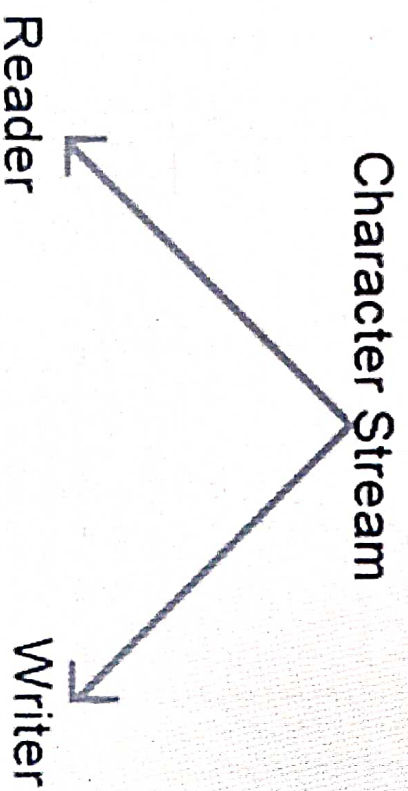
- Some important Byte stream classes
 - ◆ `BufferedInputStream` : `BufferedOutputStream`.
 - ◆ `DataInputStream` : `DataOutputStream`
 - ◆ `FileInputStream` : `FileOutputStream`
 - ◆ `ObjectInputStream` : `ObjectOutputStream`
 - ◆ `PrintStream`

Important methods

OBJECT

Methods of InputStream	Methods of OutputStream
int available()	void close()
void close()	void flush()
void mark(int readlimit)	void write(byte[] b)
boolean markSupported()	void write(byte[] b, int off, int len)
abstract int read()	abstract void write(int b)
int read(byte[] b)	
int read(byte[] b, int off, int len)	
void reset()	
long skip(long n)	

- The character streams were designed after the byte streams to provide full support for working with Unicode characters
- Character stream is also defined by using two abstract class at the top of hierarchy, they are Reader and Writer.
- These two abstract classes have several concrete classes that handle unicode character.



- Some important Character stream classes
 - ◆ BufferedReader : BufferedWriter
 - ◆ FileReader : FileWriter
 - ◆ InputStreamReader : OutputStreamReader
 - ◆ PrintWriter

Important methods

Methods of Reader	Methods of Writer
int available()	abstract void close()
void close()	abstract void flush()
void mark(int readlimit)	void write(char [] b)
boolean markSupported()	abstract void write(char [] b, int off, int len)
int read()	void write(int b)
int read(char [] b)	void write(String str)
abstract int read(char[] b, int off, int len)	void write(String str, int off, int len)
void reset()	
long skip(long n)	

- A program is often run from the command line and interacts with the user in the command line environment. The Java platform supports this kind of interaction in two ways: through the Standard Streams and through the Console.
- The Java platform supports three Standard Streams: *Standard Input*, accessed through `System.in`; *Standard Output*, accessed through `System.out`; and *Standard Error*, accessed through `System.err`.
- These objects are defined automatically and do not need to be opened. These are static objects of class `System`.
- `System.out` and `System.err` are defined as `PrintStream` objects.

Reading Console Input

OBJECT

Object of **BufferedReader** class

BufferedReader br = new BufferedReader(new

InputStreamReader (System.in));

InputStreamReader is subclass of Reader class. It converts bytes to character.

Console inputs are read from this.

- System.in is an **InputStream** which is typically connected to keyboard input of console programs
- The **java.io.InputStreamReader** class is a bridge from byte streams to character streams. It reads bytes and decodes them into characters using a specified charset

Reading from console

OBJECT

```
char c;
```

```
BufferedReader br = new
```

```
BufferedReader(new InputStreamReader(System.in));
```

```
System.out.println("Enter characters, 'q' to quit.");
```

```
// read characters
```

```
do {
```

```
    c = (char) br.read();
```

```
    System.out.println(c);
```

```
} while(c != 'q');
```

```
}
```

Reads a single

character at a

time

```
-----  
BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
```

```
System.out.print("Please enter your name? ");
```

```
String name = reader.readLine();
```

```
System.out.println("Your name is: " + name);
```

Reading strings

- Following classes can be used for reading a file
 - ◆ FileInputStream
 - ◆ FileReader
 - Following stream classes can be used for writing to file
 - ◆ FileOutputStream
 - ◆ FileWriter
 - All the classes can take file path as argument or File instance as argument.
 - File class is used for getting meta information about the file
 - All the reading streams may throw FileNotFoundException if file is not found but all the writing streams will create a file if file is not found.
- Note: only file can be created in the specified path but if path is not available it will throw the exception

- The File object represents the actual file/directory on the disk.
- Java File class represents the files and directory pathnames in an abstract manner.
- This class is used for creation of files and directories, file searching, file deletion, etc.
- There are many helper methods which can be used to manipulate the files.
 - ◆ `public boolean isFile()`
 - ◆ `public boolean exists()`
 - ◆ `public String getAbsolutePath()`
 - ◆ `public long lastModified()`
 - ◆ `public long length()`
 - ◆ `public String[] list()`



Get information about file

```
//accept file name or directory name through command line args
String fname =args[0];

//pass the filename or directory name to File object
File f = new File(fname);

//apply file class methods on File object
System.out.println("File name :"+f.getName());
System.out.println("Path: "+f.getPath());
System.out.println("Absolute path: " +f.getAbsolutePath());
System.out.println("Parent:"+f.getParent());
System.out.println("Exists :"+f.exists());
if(f.exists())
{
    System.out.println("Is writeable:"+f.canWrite());
    System.out.println("Is readable"+f.canRead());
    System.out.println("Is a directory:"+f.isDirectory());
    System.out.println("File Size in bytes "+f.length());
}
}
```


- Java `FileInputStream` class obtains input bytes from a file. It is used for reading byte-oriented data (streams of raw bytes) such as image data, audio, video etc.

- You can also read character-stream data. But, for reading streams of characters, it is recommended to use `FileReader` class.

```
File file = new File("C:/robots.txt");
FileInputStream fis = null;

try {
    fis = new FileInputStream(file);

    System.out.println("Total file size to read (in bytes) : "
        + fis.available());

    int content;

    while ((content = fis.read()) != -1) {
        // convert to char and display it
        System.out.print((char) content);
    }

    catch (IOException e) {
        e.printStackTrace();
    } finally {
        try {
            if (fis != null)
                fis.close();
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

- Note : File path can be specified using either forward slash(/) or using double backward slash(\\)
- Generally resources are closed in finally block. But from java 7 we can use :

```
File file = new File("C:/robots.txt");
```

```
try (FileInputStream fis = new FileInputStream(file)) {
```

```
    System.out.println("Total file size to read (in bytes) : "+ fis.available());
```

```
    int content;
```

```
    while ((content = fis.read()) != -1) {
```

```
        // convert to char and display it
```

```
        System.out.print((char) content);
```

```
    }
```

```
} catch (IOException e) {
```

```
    e.printStackTrace();
```

```
}
```


- Java `FileOutputStream` is an output stream used for writing data to a file.
- If you have to write primitive values into a file, use `FileOutputStream` class. You can write byte-oriented as well as character-oriented data through `FileOutputStream` class.
- But, for character-oriented data, it is preferred to use `FileWriter` than `FileOutputStream`.
- In the constructors of both the classes some constructors are accepting boolean value which indicates whether file contents are should be appended or not

Writing to file

OBJECT

```
public static void main(String args[])throws IOException {  
    File file = new File("Hello1.txt");  
  
    // creates the file  
    file.createNewFile();  
  
    // creates a FileWriter Object  
    FileWriter writer = new FileWriter(file);  
  
    // Writes the content to the file  
    writer.write("This\n is\n an\n example\n");  
    writer.flush();  
    writer.close();  
  
    // Creates a FileReader Object  
    FileReader fr = new FileReader(file);  
    char [] a = new char[50];  
    fr.read(a); // reads the content to the array  
  
    for(char c : a)  
        System.out.print(c); // prints the characters one by one  
    fr.close();  
}
```